

EXPERIMENT NO. 1

Develop a semantic, multi-page technical documentation site

Name: Rohit Pujari

Roll No.: 28

Date: _____

Aim:

To develop the documentation website of “RailTrack”, a practice REST API for Indian Railways timetables, as four linked pages (Overview, Getting Started, Endpoints and Errors & FAQ) written in semantic HTML5 with landmarks, breadcrumbs, a skip link, accessible data tables, definition lists and disclosure widgets, and to check every page automatically with a script that tests the heading outline, the landmarks, the table headers and all internal links.

Theory:

Semantic HTML means choosing each element for what the content is, not for how it should look. A `<div>` or `<span>` says nothing about its content, while `<nav>`, `<main>`, `<table>` or `<dl>` tell the browser that the content is a menu, the main part of the page, tabular data or a list of terms. Browsers build an accessibility tree from these meanings, which screen readers use to jump between regions and headings; search engines and reader modes use the same information. The look is added separately with CSS, so the markup stays clean and easy to maintain.

Page structure and landmarks

Element	Use in the RailTrack docs
<code><header></code>	Site banner with the logo link and the API version written as <code><data value="2.1"></code>
<code><nav aria-label="Documentation"></code>	Side menu of the four pages; the current page has <code>aria-current="page"</code>
<code><nav aria-label="Breadcrumb"></code>	An ordered list (Docs / Endpoints) showing where the page is; the two <code><nav></code> elements are told apart by their <code>aria-label</code>
<code><main id="content"></code>	The one main region of each page; target of the skip link
<code><section aria-labelledby></code>	Each section is named by the id of its own <code><h2></code>
<code><aside></code>	A side note about rate limits that is related to, but not part of, the main text
<code><footer></code> , <code><address></code> , <code><time></code>	Contact of the maintainers and a machine-readable update date

Content elements chosen for documentation

- **`<dl>`, `<dt>`, `<dd>` and `<dfn>`:** name-value pairs such as the key terms and the status codes; `<dfn>` marks the place where a term is defined.
- **`<ol>` versus `<ul>`:** the setup steps use an ordered list because the order matters; the list of possible applications uses an unordered list.
- **`<figure>`, `<pre>`, `<code>`, `<samp>`, `<kbd>`:** code listings are figures with a `<figcaption>`; `<code>` is code to type, `<samp>` is output from the program and `<kbd>` is a key on the keyboard.
- **`<abbr title>`:** gives the full form of REST, JSON and UTC when the pointer rests on the word.
- **`<details>` and `<summary>`:** each FAQ answer is a native disclosure widget that opens with mouse or keyboard without any JavaScript. The link `faq.html#limits` opens the page at the matching answer.

Accessible data tables

The endpoint table has a `<caption>`, column headers in `<thead>` with `scope="col"` and the first cell of each row written as `<th scope="row">`. A screen reader can then announce every cell together with its row and column header, for example “GET, Path, /trains/{number}”.

Headings, skip link and metadata

Every page has exactly one `<h1>`, `<h2>` for its sections and no skipped levels, so the heading outline is a correct table of contents. A “Skip to content” link is the first focusable element; it is moved off-screen with CSS and appears when it receives keyboard focus. Each page has `lang="en"`, its own `<title>` and a meta description for search results.

Consistent pages and automatic checks

The small script `make_pages.py` writes the four pages from one layout function, so the header, menu and footer are identical everywhere and `aria-current` is set on the correct menu item automatically. The script `check.js` opens every page in Chromium through Playwright and tests the rules below.

Check	Rule
Language and metadata	<code>lang="en"</code> and a meta description are present
Headings	Exactly one <code><h1></code> ; no level is skipped (for example <code><h2></code> followed by <code><h4></code>)
Landmarks	<code><header></code> , <code><nav></code> , <code><main></code> and <code><footer></code> all exist; the side menu has <code>aria-current</code>
Sections and tables	Every <code><section></code> has <code>aria-labelledby</code> ; every <code><th></code> has a scope
Links	Every internal link points to an existing page and, with <code>#id</code> , to an existing id

Result of `node check.js`: all four pages passed, with 7 internal links per page and no broken link.

Important Code:

HTML - page skeleton (index.html)

```
<body>
  <a class="skip" href="#content">Skip to content</a>
  <header class="top">
    <a class="brand" href="index.html">RailTrack <span>API Docs</span></a>
    <p class="version">Version <data value="2.1">v2.1</data></p>
  </header>
  <div class="layout">
    <nav class="side" aria-label="Documentation">
      <ul>
        <li><a href="index.html" aria-current="page">Overview</a></li>
        <li><a href="getting-started.html">Getting Started</a></li>
        <li><a href="endpoints.html">Endpoints</a></li>
        <li><a href="faq.html">Errors & FAQ</a></li>
      </ul>
    </nav>
    <main id="content">
```

HTML - accessible table (endpoints.html)

```
<table>
  <caption>Table 1: Endpoints of RailTrack API v2</caption>
  <thead>
    <tr><th scope="col">Method</th><th scope="col">Path</th><th scope="col">Returns</th></tr>
  </thead>
  <tbody>
    <tr><th scope="row">GET</th><td><code>/trains/{number}</code></td><td>Timetable of one
train</td></tr>
    ...
  </tbody>
</table>
```

HTML - FAQ entry and CSS for the current page

```
<details id="limits" open>
  <summary>What are the rate limits?</summary>
  <p>100 requests per minute on the free plan and 1000 on the college plan.</p>
</details>

.side a[aria-current="page"] { border-left-color: #0f766e; color: #0f766e; font-weight: 600;
background: #f0fdfa; }
.skip { position: absolute; left: -999px; }
.skip:focus { left: 12px; top: 8px; background: #fff; padding: 6px 10px; border: 2px solid
#0f766e; }
```

Output:

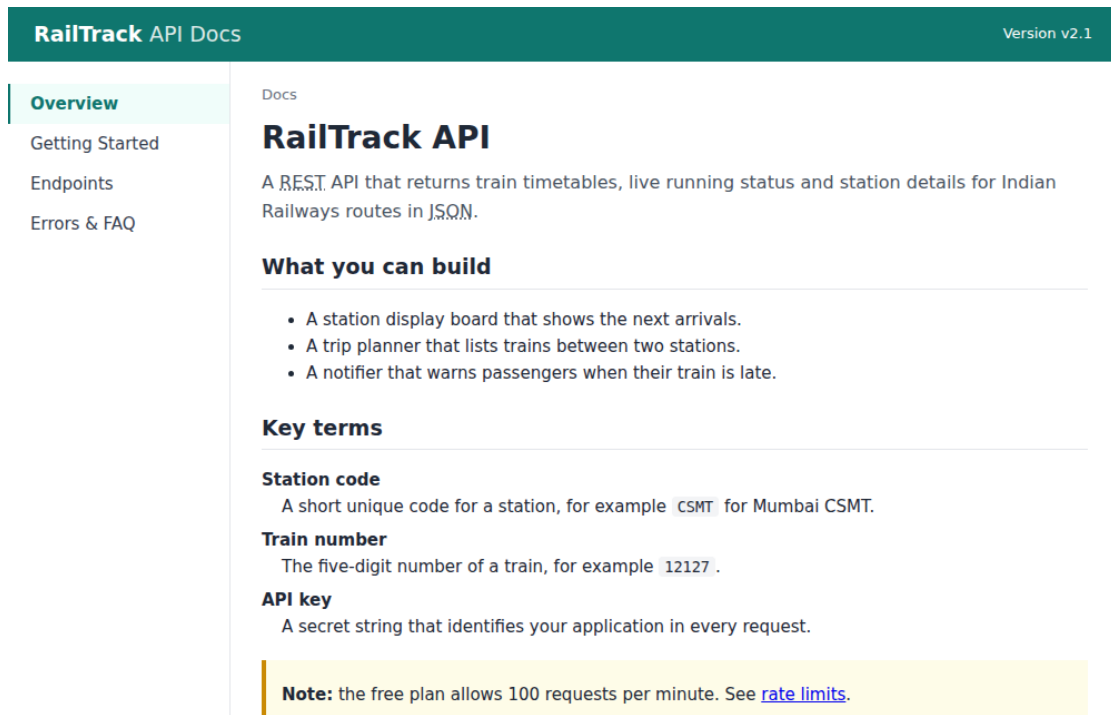


Figure 1.1: Overview page - side menu with aria-current, breadcrumb, <abbr>, <dl> with <dfn> and an <aside> note

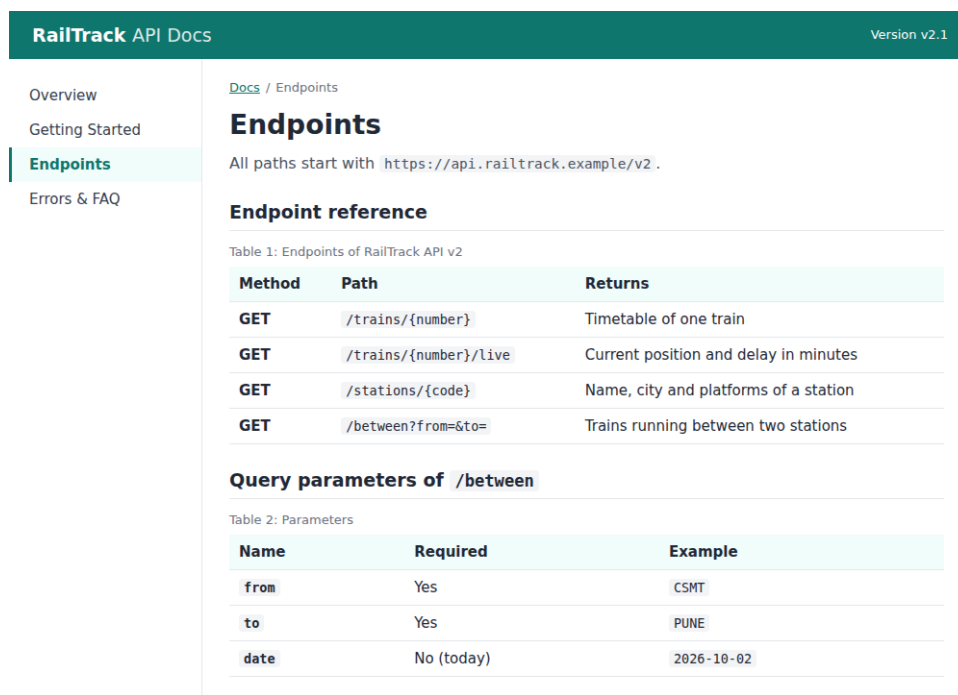


Figure 1.2: Endpoints page - tables with <caption>, column headers and row headers (th scope="row")

Conclusion:

A four-page technical documentation site was developed in semantic HTML5. Landmarks, breadcrumbs, a skip link and a correct heading outline made the pages easy to navigate, while definition lists, figures, accessible tables and <details> widgets gave each kind of content its proper meaning. The automatic check script confirmed that all four pages follow these rules and have no broken links.